

Weeks 2-10

Extra / grades 6-7

Optional investigations; offer the next page by readiness.

Grade bands are entry points. Begin with objects, then try to explain what happens.

Give one page at a time. The packet is a menu, not a checklist for one hour.

Keep facilitator solutions separate from student pages.

Week and investigation	PDF pages
02 Switches and lamps	2-4
03 Repeating secret machines	5-7
04 Around the ring	8-10
05 Tower cities	11-12
06 Code-breaking with guarantees	13-15
07 Take-away games	16-18
08 Rooks and piles	19-21
09 Bouncing paths	22-24
10 Bridges and delivery routes	25-27

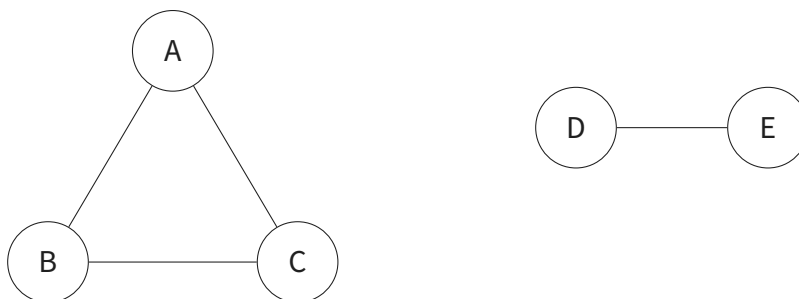
Print US Letter, single-sided, at 100% / Actual Size.

Page ranges above count this cover as page 1. Bookmarks open each week.

Weekly footers retain their own page numbers and activity IDs for the use log.

Revised September 27, 2026. Unpiloted revisions; record actual use after teaching.

Problem 1: Place a two-sided counter on every lamp, all OFF. Press an edge to flip both ends. Start over for each target in the table. Record moves that work, or “not yet.” A move cannot jump across a gap.



ON lamps	Moves, or not yet
A,C	
D,E	
A,D	
A,C,D,E	
A,B,C,D	
A,B,D,E	

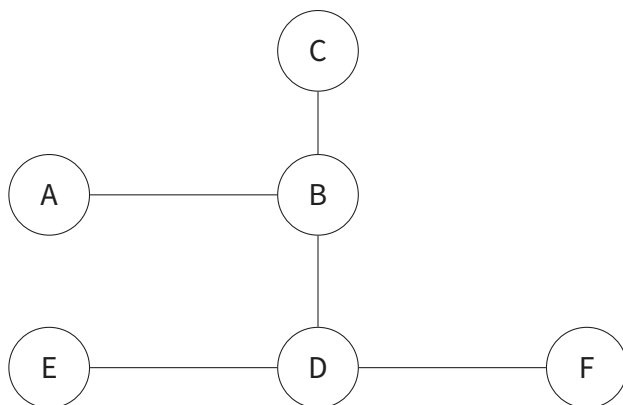
Problem 2: Count the target lamps in each connected piece separately. Sort your targets using those two counts. Record a rule that predicts which targets work.

Problem 3: Add a new edge CD to the page-1 network. Try the two formerly unsuccessful targets A,D and A,B,C,D again. Record your paths and resulting move lists.

Problem 4: On the joined network, use paths A–C–D and B–C–D to make target A,B. Try the full list including the repeated CD. Then remove both CD presses and replay it. Compare the final lamps.

Problem 5: Draw a different network with two connected pieces, at least three lamps in each. Choose one target your rule predicts can be made and one it predicts cannot. Try both and record them on your drawing.

Problem 6: A tree is a connected network with no closed loop. On this tree, start all OFF and make target A,C. Reset and make A,C,E,F. Find a reduced move list for each: use every edge at most once.



Two move lists:

Problem 7: For target A,C,E,F, imagine removing each edge. Count the targets on the named side of the cut. Compare odd/even with whether your move list uses that edge.

Cut edge	Count on this side	Target count	Edge used?
AB	A		
BC	C		
BD	A,B,C		
DE	E		
DF	F		

Problem 8: Choose a different even target. Use cut counts to predict its edge set before testing. Draw or write the prediction, then replay it.

Problem 1: Use cards A–H to make a reversible key: each letter has one output, and each output is used once. Build the four sets of loops below. Move a counter one arrow per turn on each loop. Record the first turn when all counters are home.

Loop sizes	My key or arrow loops	Return
8		
4 + 4		
6 + 2		
5 + 3		

Problem 2: For the 5+3 key, list the return turns of each separate loop through turn 20. Circle the first turn on both lists. Repeat for 4+4. Compare with one loop of size 8.

Problem 3: Find the slowest eight-letter key. Sort possible loop sizes by the largest loop. Use eight cards to check that your sizes total eight. For largest loops 4–8, list the splits of the leftover cards and find the best return. In the last row, use loop sizes 1,2,3 to find one turn that returns every loop. A fixed letter is a loop of size 1.

Largest loop	Possible leftover loops	Best return
8	none	
7		
6		
5		
4		
At most 3		

Problem 4: Compare the best return in every row with your page-1 key. Record the winning loop sizes, key, and return time.

Problem 5: Add card I. Build keys with loops $5+4$, $7+2$, and $4+3+2$. List each loop's return turns until they first agree. Record the whole-key return for each.

Problem 6: Classify nine-letter keys by largest loop, as on page 2. Find the best return for largest loops 9,8,7,6,5,4. When all loops have size at most 3, give one turn that returns all of them. Compare these bounds with your constructions.

Problem 7: Use six cards. Test loops $3+2+1$, 6, and $5+1$. Compare with the five-card $3+2$ key. Predict whether adding a card must always make the longest possible return strictly larger. Record a supporting example and a question still to check.

Problem 1: Lay out cards 0–7, leftmost first. Cut into two equal rows without reversing either half. Interleave one card at a time, starting with the first half. The first shuffle is shown. Rebuild it with cards, then repeat the same rule until the original row first returns. Record every row.

Shuffle	Card labels, left to right
0	0 1 2 3 4 5 6 7
1	0 4 1 5 2 6 3 7
2	
3	
4	

Problem 2: Number places 0–7 from left to right on a strip of paper. Put the original row above it. Track card 1 through the shuffles, recording its place each time. Reset and track card 3, then card 7.

Problem 3: Reset to 0–7. Shuffle once. Fill where each old position moves. A card’s label stays the same; its position can change. For example, the card at old position 4 moves to position 1.

Old position	0	1	2	3	4	5	6	7
New position					1			

Problem 4: Draw arrow loops for this position map, starting at 0 and then at each unused position. Use the loops to predict the first return of the whole row. Check against page 1.

Problem 5: For positions 0–6, double the old position and subtract 7 if needed to leave a number from 0 to 6. Compare with your measured new positions. Treat position 7 separately.

Old position	0	1	2	3	4	5	6
Double							
Remainder after 7							

Problem 6: Starting with 1, repeatedly double and keep the remainder after division by 7. Record until 1 returns. Compare with the route of old position 1.

Problem 7: Use six cards 0–5. Repeat the same out-shuffle until the full row returns. Record rows. Then test whether the position rule is doubling with remainder after division by 5, with position 5 fixed.

Problem 8: For sixteen cards 0–15, predict the return by starting with 1, repeatedly doubling, and taking the remainder after division by 15. Check with cards or draw the position loops; keep position 15 fixed separately.

Problem 9: Compare the first return times for 6,8,16 cards. Record one comparison showing whether a larger deck must take more shuffles to return.

Problem 1: Copy this city. Every row and column must contain 1–4 once. In the upper-left bold block, exchange the 1s and 2s. Record the changed city on the right, then check each row and column.

1	2	3	4
2	1	4	3
3	4	1	2
4	3	2	1

Problem 2: Start from the original city each time. Switch the two heights within just one other bold block. Record each changed city below. Check every row and column after each switch.

Upper right

Lower left

Lower right

Rows and columns affected by my lower-left switch:

Problem 3: An entry clue fixes the height in one cell. There are no outside clues on this page. Copy the original city from Problem 1. Circle three cells as clues. Choose a bold block with no circled cell. Switch its two heights, then check that all three clues still hold. Repeat using a different choice of three clues.

Problem 4: Now keep the four entries shown below. They touch all four bold blocks. Build two different cities that keep these entries and use 1–4 once in every row and column.

1		3	
3		1	

1		3	
3		1	

Cells I changed while keeping the four clues:

Problem 1: Use five counters in a row. Write R as 0 and B as 1. A score counts matching positions. Build each secret below and score all four tests. Keep the secret visible while checking.

Secret	00000	00011	00101	01001
00000				
11111				
10010				

Problem 2: Choose a new secret and hide it. Your partner uses exactly the same four tests. Record the scores. Reveal the secret and check each score. Then swap jobs.

Round 1		Round 2	
Test	Score	Test	Score
00000		00000	
00011		00011	
00101		00101	
01001		01001	

Revealed secrets:

Problem 3: Change two test positions from 00 to 11 while leaving all other positions fixed. Build each possible secret pair below. Count its matches with 00 and with 11, and record the change.

Secret pair	Matches with 00	Matches with 11	Score change
00			
01			
10			
11			

Problem 4: For one hidden five-place secret, tests 00000, 00011, 00101, and 01001 scored 2, 2, 2, and 0. Use the first score to count its 1s. Compare each later score with the first to find the number of 1s in each changed pair.

Positions	Number of 1s
All five positions	
Positions 4 and 5	
Positions 3 and 5	
Positions 2 and 5	

Problem 5: Use those counts to build the secret. Record it and check all four scores directly.

My secret and its four scores:

Problem 6: Try to build a secret with exactly one 1 in each of these pairs: positions 4 and 5, positions 3 and 5, positions 2 and 5. First put 0 in position 5, then put 1 there. In each case try both choices for position 1. Record all four words and their total numbers of 1s.

Position 5	Position 1	Five-place word	Total 1s
0	0		
0	1		
1	0		
1	1		

Problem 7: Score your four words using the four tests from Problem 1. Compare the score records. Circle a score that separates each pair of words.

Secret	00000	00011	00101	01001

Problem 8: Return to your two hidden secrets from Problem 2. Use the total number of 1s and the three pair counts to recover them from the scores alone. Record the recovered words.

Recovered words:

Problem 1: Use two piles. On a turn, take 1, 3, or 4 counters from exactly one pile; do not take more than that pile holds. Players alternate. Taking the very last counter wins. Play each starting pair below. Record the two pile sizes after every move, writing (left pile, right pile).

Start	Pairs after each move	Winner: first or second
(1,1)		
(1,3)		
(1,4)		
(4,4)		

Problem 2: Play again from (1,4). Try every legal first move from either pile. Reset before each attempt. Record which moves leave a position your opponent cannot win against your replies.

First moves and the pairs they leave:

Problem 3: Give each single pile a number label. Set $g(0)=0$. For a larger pile, list the labels of every pile reachable by taking 1, 3, or 4. Its g -label is the smallest nonnegative integer missing from that list. For pile 1, the reachable label is 0, so $g(1)=1$. For pile 2, it is 1, so $g(2)=0$. Build these small cases, then complete the chart in order.

Pile	Legal next piles	Their g -labels	Smallest missing
0	none	none	0
1	0	0	1
2	1	1	0
3			
4			
5			
6			
7			
8			
9			
10			

Problem 4: Choose piles 4 and 5. Build every legal move from each. Use your chart to record the labels reached. Circle any nonnegative label smaller than the starting label that you can reach.

Starting labels and reachable labels:

Problem 1: Use two piles. On a turn, remove one or more counters from one pile OR remove the same positive number from both. Take the last counter to win. Play each start twice, changing who starts. Record one first move and reply.



Start	First move leaves	Reply leaves
(2,2)		
(1,2)		
(2,3)		
(3,5)		

Problem 2: From (1,2), try every legal first move. There are four. Draw or list them, then find a reply that wins immediately in each case.

Problem 3: Make a map to find starts worth giving your opponent. T means its next player loses against perfect replies; W means that player can force a win. Start with T at (0,0), where play is over. Work in increasing $a+b$. Mark W when you can move to a T. Mark T only when all legal moves go to W positions. Pile order does not matter; sort the new sizes before locating a move in this table.

$a \setminus b$	0	1	2	3	4	5	6	7
0	T							
1	–							
2	–	–						
3	–	–	–					
4	–	–	–	–				
5	–	–	–	–	–			
6	–	–	–	–	–	–		
7	–	–	–	–	–	–	–	

Problem 4: List the T pairs you found. For each of (2,4), (4,6), and (5,7), build the piles and find a legal move to one of your T pairs. Record the moves.

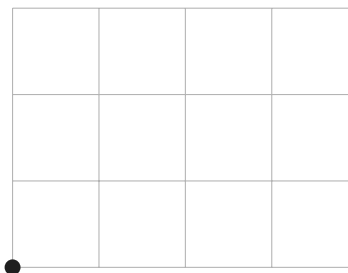
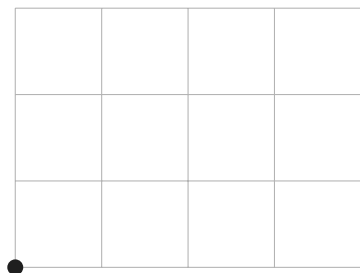
Problem 5: Build a second list. Begin with (0,0). In row n , choose the smallest nonnegative number not yet used in either column as a_n . Set $b_n = a_n + n$. Complete the table, crossing numbers off the strip as you use them. Compare its pairs with your game map.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
n	0			1				2			3		4		5
a_n	0			1											
b_n	0			2											
$b_n - a_n$	0			1											

Problem 6: Choose two pairs in your list. Try to move from the larger pair to the smaller by taking from one pile, then by taking equally from both. Repeat for two other pairs. Record what prevents each move, or record a move if you find one.

Problem 7: A known theorem gives the losing pairs as $(\lfloor n\phi \rfloor, \lfloor n\phi^2 \rfloor)$, where $\phi = (1 + \sqrt{5})/2 \approx 1.618034$. The brackets mean round down. Use a calculator to check $n=1,2,3,4,5$ against your list. Then find moves into the list from (5,8) and (8,12). Record the moves.

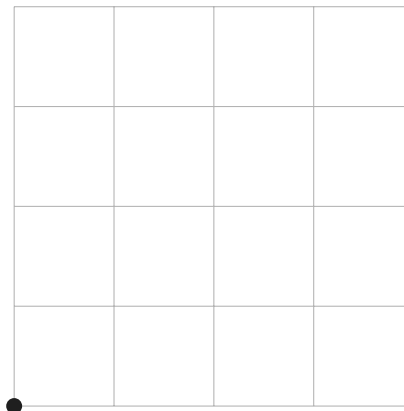
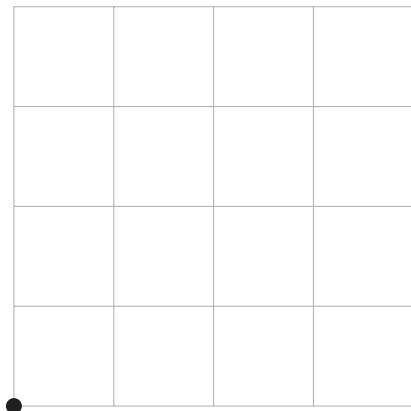
Problem 1: Use a unit square, unfolded into the square grids below. Each little square is one whole reflected table. Start at the black dot. Slope means vertical distance divided by horizontal distance. Draw slopes $1/2$, $2/3$, and $3/2$, one on each grid. Stop at the first grid vertex after the starting dot. Mark each wall crossing before it.



Problem 2: Record each first vertex as (horizontal distance, vertical distance). Fold the whole-room counts back to predict the corner. Count the wall crossings before it.

Slope	First vertex	Corner	Bounces
$1/2$			
$2/3$			
$3/2$			

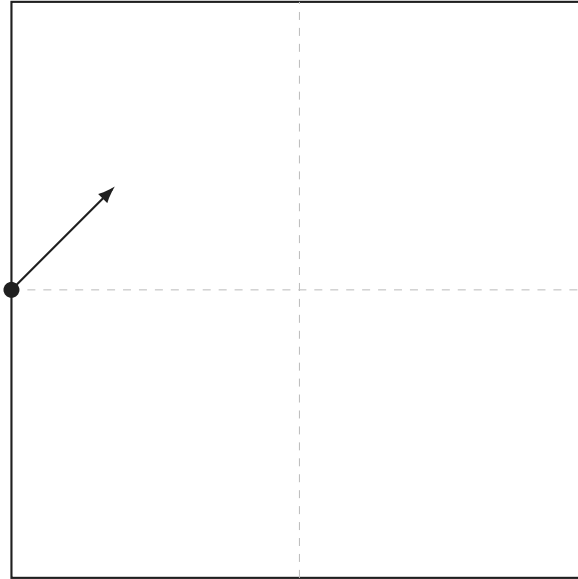
Problem 3: On the left grid draw the line of slope $2/4$. On the right draw the line of slope $3/4$. Stop each at its first grid vertex. Record the vertex, corner, and bounces; compare $2/4$ with your $1/2$ path.



Problem 4: Use positive fractions p/q with no common factor between p and q . Find a slope whose path ends top-right after four bounces. Find one that ends bottom-right after three bounces. Record each slope and its first unfolded vertex, then check the wall counts.

Problem 5: Now use slope $\sqrt{2}$. If its unfolded path from $(0,0)$ passed through a positive integer point (u,v) , its slope would be v/u . Record what this would say about $\sqrt{2}$. You may use the fact that $\sqrt{2}$ is not a ratio of integers.

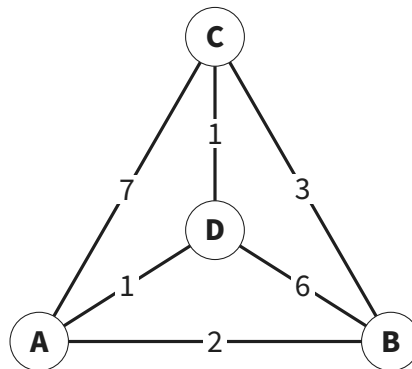
Problem 6: Start at the midpoint of the left wall of this unit square, heading up-right with slope 1. Trace four bounces. Continue for four more. Mark a point inside the square that your path never reaches.



Problem 7: Compare your midpoint path with the slope- $\sqrt{2}$ path from $(0,0)$. For each, record whether it can reach a corner. For the midpoint path, draw a small region that the path never enters.

Problem 8: A known theorem says the slope- $\sqrt{2}$ path from $(0,0)$ comes arbitrarily close to every point in the square. This is called dense. Circle which evidence below establishes density: “I did not hit a corner” / “the stated theorem.” Use your midpoint path to give an example showing why the other claim is insufficient.

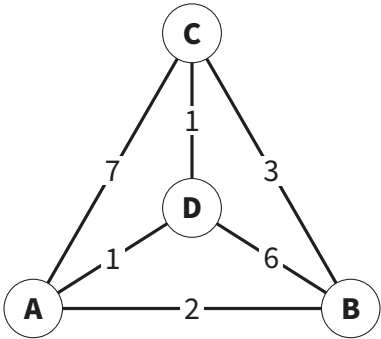
Problem 1: Start at A, travel every road at least once, and return to A. Numbers are road lengths; every repeat costs that length again. Find two complete routes. Record each visited dot and add the traveled lengths. Circle your shorter route.



Trial	Route	Length
1		
2		

Problem 2: Add the lengths of the six original roads. In one of your routes, mark the first trip along each road. List only the extra trips and their total cost. Even a long road must be traveled once.

Problem 3: Find the cheapest connection between each pair of dots. A connection may follow several roads. Compare the direct road with a route through another dot; record the cheapest route and its cost.

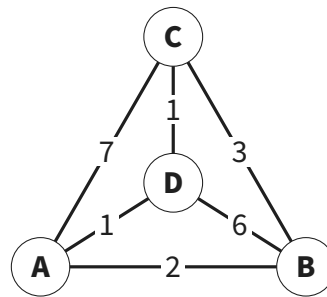
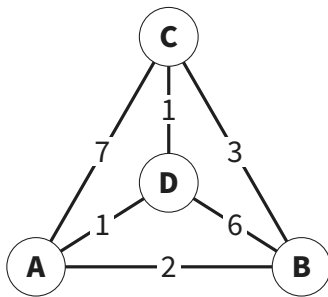


Pair	Direct cost	Cheapest connection	Cost
AB	2		
AC	7		
AD	1		
BC	3		
BD	6		
CD	1		

Problem 4: The four dots all have odd degree. Pair them in each way below and add the two cheapest connection costs. Compare the extra costs. Circle the best pairing.

Pairing	Total extra cost
AB + CD	
AC + BD	
AD + BC	

Problem 5: On the left map mark extra trips for your best pairing. On the right mark extra trips for $AD + BC$. Find a closed route on each map using every original and extra road copy exactly once. Record both routes and total lengths.



Problem 6: A proposed rule is “Choose a cheapest pair first, then connect the remaining two dots.” Choose AD first and calculate the result. Choose CD first and calculate the result. Record whether the rule always finds the best total in this example.

Problem 7: In your best route, erase one required copy of each original road from its crossing list. Draw just the extra road copies below. Circle their odd-degree endpoints and match them with one of the pairings in Problem 4.